



南開大學
Nankai University

计算机学院
信息检索系统原理实验报告

NKU Web Search Engine

姓名：赵曼欣
学号：2312063
专业：计算机科学与技术

2025 年 6 月 11 日

目录

1 项目简介	2
2 实验环境	2
3 具体功能实现	3
3.1 网页抓取	3
3.2 文本索引	3
3.3 查询服务	4
3.3.1 站内查询	4
3.3.2 文档查询	6
3.3.3 短语查询	7
3.3.4 通配查询	9
3.3.5 查询日志	10
3.3.6 网页快照	12
3.4 个性化查询	13
3.5 个性化推荐	15
3.6 web 界面	16
3.6.1 整体架构	16
3.6.2 核心功能模块	16

1 项目简介

本项目是针对南开校内的新闻资源构建的一个 Web 搜索引擎，为用户提供南开新闻的查询服务和个性化推荐，主要包含网页抓取、文本索引、查询服务、个性化查询和推荐四个功能。

2 实验环境

- 框架和工具：

Elasticsearch：提供数据存储、检索与排序等功能；

Flask：实现 Web 应用和后端逻辑；

BeautifulSoup：用于网页内容解析；

txt 和 JSON 文件：保存用户数据与查询历史记录

- 实验目录结构

```
CODE/
├── index/
│   ├── index_config.py
│   ├── doc_importer.py
│   └── data_importer.py
├── query/
│   ├── templates/
│   │   ├── search.html
│   │   ├── results.html
│   │   ├── history.html
│   │   ├── snapshot.html
│   │   ├── login.html
│   │   ├── register.html
│   │   └── user_home.html
│   └── query.py
├── static/
│   └── style.css
├── data/
│   ├── output_多彩校园/
│   ├── output_光影南开/
│   └── ...
├── data_doc/
├── doc/
├── snapshots_1/
├── spider.py
├── spider_doc.py
├── change.py
├── query_log.json
├── users.json
└── cache/
```

数据索引与导入模块
定义并创建索引
文档批量导入ES脚本
网页批量导入ES脚本
查询与web应用模块
Flask HTML模板
搜索页面
结果页面
查询历史页面
快照页面
登录页面
注册页面
用户主页
Flask主程序
静态资源
样式文件
爬取的网页数据
爬取的文档数据
附件原始文件（doc/pdf/xls等）
网页快照HTML
网页爬虫程序
文档爬虫程序
删除索引
查询日志
用户信息
缓存目录

3 具体功能实现

3.1 网页抓取

- 利用 spider 文件实现的网页抓取和数据保存，其中主要对南开新闻网和部分学院官网进行爬取，包括 ["https://news.nankai.edu.cn", "https://www.nankai.edu.cn", "https://cc.nankai.edu.cn", "https://ai.nankai.edu.cn", "https://cs.nankai.edu.cn"] 等网站，通过 spider.py 文件爬取网页信息和保存相关快照，通过 spider_doc.py 文件爬取文档信息
- 支持同域名网页爬取，将抓取到的网页内容保存为 txt 文件进行存储，遵守 robots.txt 协议，礼貌抓取网页资源。
- 爬取内容示例：

Title: 马克思主义学院第六届“公能杯”学生学术论坛举办

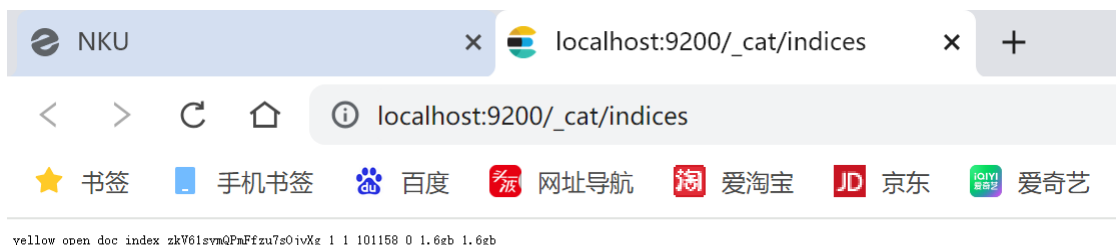
URL: http://news.nankai.edu.cn/dcxy/system/2025/06/01/030067406.shtml

Date: 2025-06-01

News Snapshot Hash: 9a7769e49d62c1e7ce5b2b1c3b698ad4

Anchor texts: ["南开大学", "多彩校园"]

- 101158 为爬取的数量，符合要求，如下图所示：



3.2 文本索引

通过 index_config.py 文件定义 Elasticsearch 的索引映射和分词，具体结构如下，可以满足要求中的各类查询功能：

标准查询：title、url、content、anchor_texts 字段

文档查询：type、filename 字段

快照功能：snapshot_hash、date 字段

索引结构

```
1 "mappings": {
2   "properties": {
3     "url": {"type": "keyword", "ignore_above": 256},
```

```
4         "title": {
5             "type": "text",
6             "analyzer": "my_max_analyzer",
7             "index_options": "offsets"
8         },
9         "content": {
10             "type": "text",
11             "analyzer": "my_max_analyzer",
12             "index_options": "offsets"
13         },
14         "anchor_texts": {
15             "type": "text",
16             "analyzer": "my_max_analyzer",
17             "index_options": "offsets"
18         },
19         "date": {"type": "date", "format":
20             "yyyy-MM-dd||yyyy/MM/dd||epoch_millis"},
21         "snapshot_hash": {"type": "keyword"},
22         "snapshot_text": {
23             "type": "text",
24             "analyzer": "my_max_analyzer"
25         },
26         "source_file": {"type": "keyword"},
27         "type": {"type": "keyword"},
28         "filename": {"type": "keyword"}
29     }
30 }
```

- 此处我未将快照中的 html 代码内容进行构建索引, 防止 C 盘爆炸, 爬取网页数据的同时爬取快照 hash 以建立快照 hash 和对应数据集的联系, 用于查找网页快照。
- 对日期字段进行标准化处理, 统一存储为 datetime 格式。
- 安装 ik 分词器进行中文分词, 通过 data_importer.py 文件和 doc_importer.py 文件对爬取的数据进行处理, 将存储在 txt 文件里的网页数据导入到 elasticsearch 中。

3.3 查询服务

通过 query.py 文件实现查询服务的主要实现

3.3.1 站内查询

该部分需要实现用户能在特定网站域名, 即本项目在 `nankai.edu.cn` 相关域名内进行内容搜索, 在我的代码里通过 `site_search` 函数实现: 使用 Elasticsearch 的 `multi_match` 查询来搜索多个指定字段, 包括 `title`、`content`、`snapshot_text` 等。

代码实现

索引结构

```
1 def site_search(query_term, page=1, page_size=15):
2     body = {
3         "query": {
4             "multi_match": {
5                 "query": query_term,
6                 "fields": ["title", "content", "snapshot_text"]
7             }
8         },
9         "from": (page - 1) * page_size,
10        "size": page_size
11    }
12    return es.search(index=INDEX_NAME, body=body)
```

示例演示

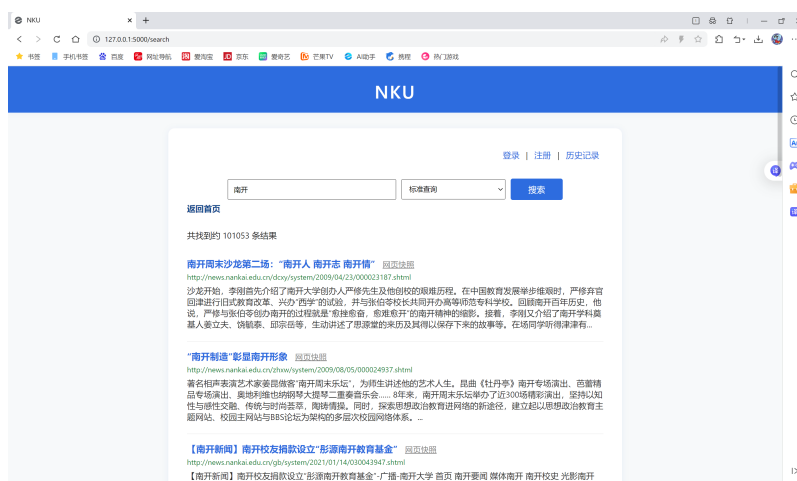


图 3.1: 站内查询

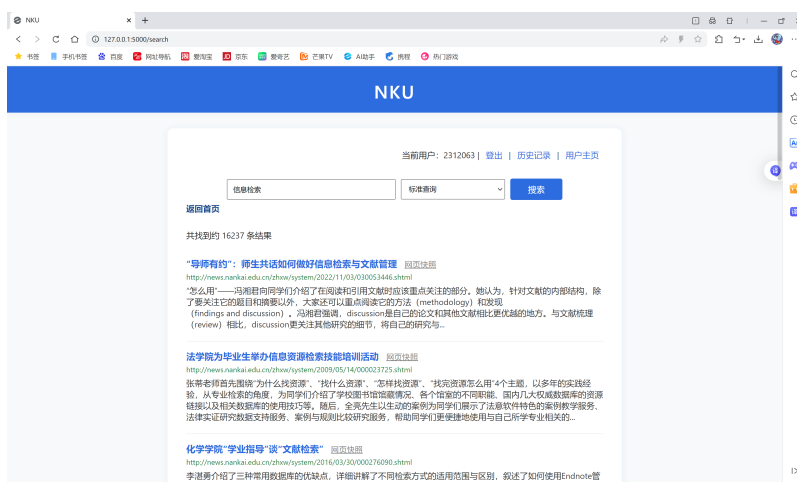


图 3.2: 站内查询

3.3.2 文档查询

一些网页可能会携带或其本身就是附件下载链接，支持对文档的查询操作。文档查询专注于查找包含特定文件类型（如 PDF、DOCX 等）的文档，通过 `document_search` 函数进行实现：首先定义了支持的文件类型列表，然后构建一个查询体，其中包括一个 `multi_match` 查询和一个 `filter` 来确保只返回指定文件类型的文档。

代码实现

索引结构

```
1 def document_search(query_term, page=1, page_size=15, filetypes=None):
2     SUPPORTED_FILETYPES = ['pdf', 'doc', 'docx', 'xls', 'xlsx']
3     if not filetypes:
4         filetypes = SUPPORTED_FILETYPES
5     field_config = get_field_config('all')
6     fields = field_config["fields"] + ["filename", "type"]
7     weights = field_config["weights"].copy()
8     weights.update({"filename": 1.5, "type": 1.0})
9
10    body = {
11        "query": {
12            "bool": {
13                "must": [
14                    {
15                        "multi_match": {
16                            "query": query_term,
17                            "fields": [f"{f}^{weights.get(f,1)}" for f in fields]
18                        }
19                    }
20                ],
21                "filter": [
22                    {"terms": {"type": [ft.lower() for ft in filetypes]}}
23                ]
24            }
25        },
26        "from": (page - 1) * page_size,
27        "size": page_size,
28        "track_total_hits": True
29    }
30    return es.search(index=INDEX_NAME, body=body)
```

示例演示

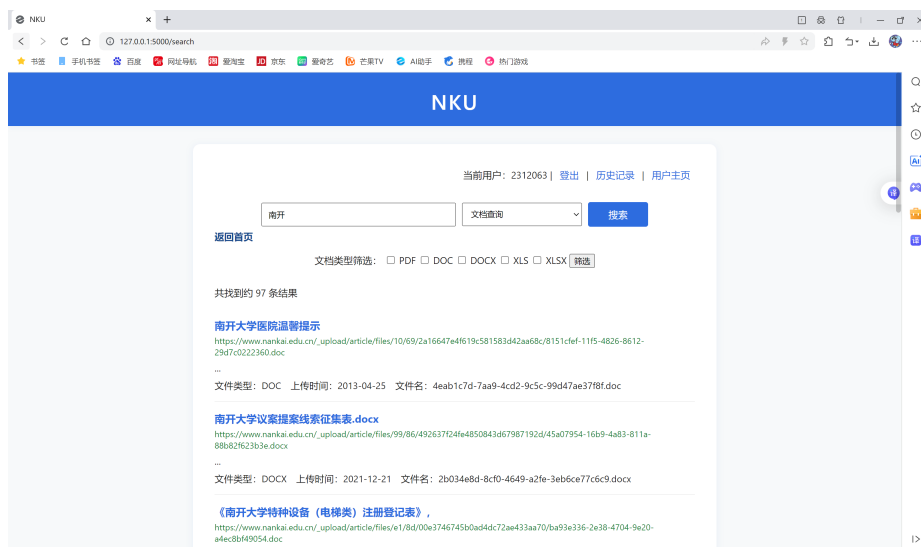


图 3.3: 文档查询

可以发现搜索的关键词不变，但选择特定文件类型后文件数量减少，成功实现筛选：

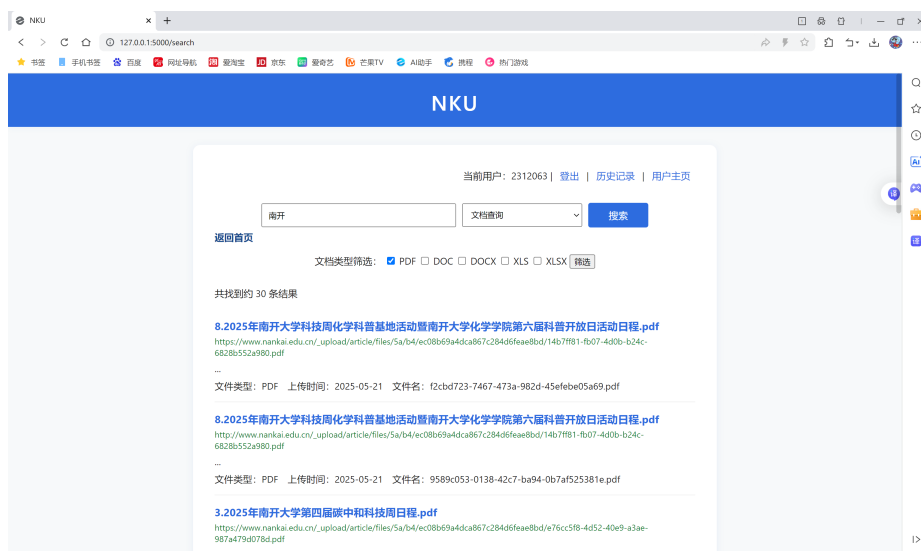


图 3.4: 文档查询

3.3.3 短语查询

短语查询支持对多个 Term 的查询，通过 `phrase_search` 函数进行实现：使用 `jieba.cut` 方法对查询词进行分词，然后为每个字段构建一个 `match` 子句，确保所有分词都出现在文档中。

代码实现

索引结构

```
1 def phrase_search(query_term, page=1, page_size=15):
2     field_config = get_field_config('all')
3     fields = field_config["fields"]
4     terms = list(jieba.cut(query_term))
5     should_clauses = []
```



```
6 for field in fields:
7     must_terms = [{"match": {field: term}} for term in terms]
8     should_clauses.append({
9         "bool": {
10             "must": must_terms
11         }
12     })
13 body = {
14     "query": {
15         "bool": {
16             "should": should_clauses,
17             "minimum_should_match": 1
18         }
19     },
20     "from": (page - 1) * page_size,
21     "size": page_size,
22     "track_total_hits": True
23 }
24 return es.search(index=INDEX_NAME, body=body)
```

示例演示

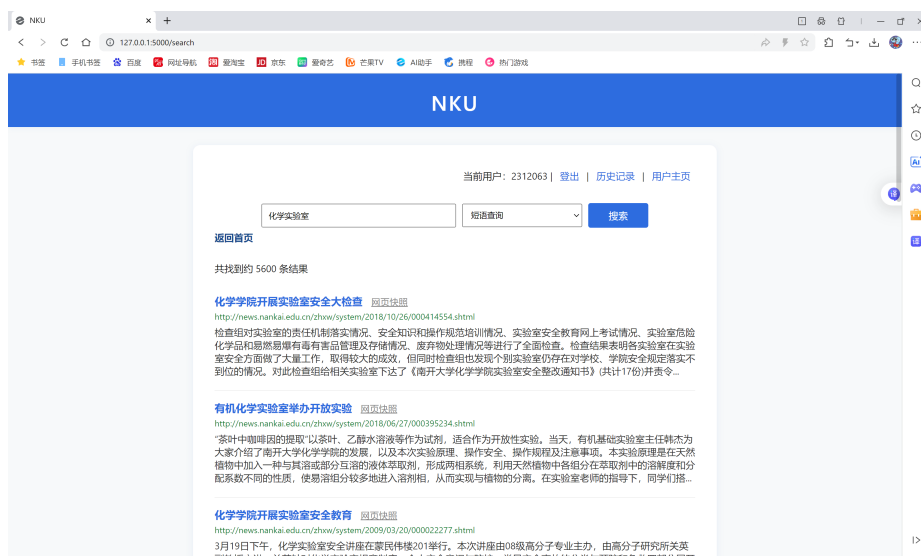


图 3.5: 短语查询

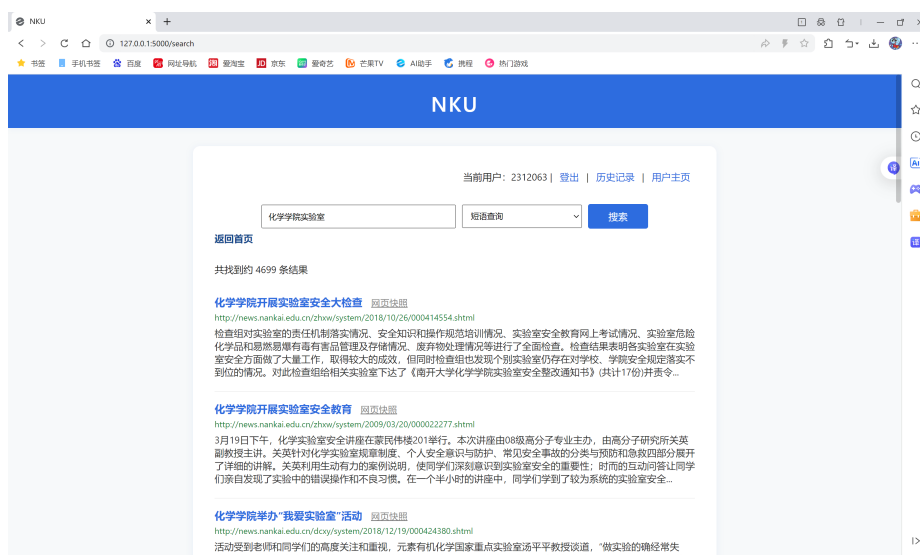


图 3.6: 短语查询

3.3.4 通配查询

通配查询允许用户使用通配符（* 和?）进行搜索，通过 `wildcard_search` 函数进行实现：处理查询词中的中文通配符，并为每个字段构建一个 `wildcard` 查询。

代码实现

索引结构

```

1 def wildcard_search(query_term, page=1, page_size=15):
2     field_config = get_field_config('all')
3     fields = field_config["fields"]
4
5     query_term = query_term.replace('?', '?').replace('*', '*')
6     if query_term.strip('*?') == '':
7         return {"hits": {"total": {"value": 0}, "hits": []}}
8
9     should_clauses = [
10         {"wildcard": {"field": {"value": f"*{query_term}*"}}}
11         for field in fields
12     ]
13     body = {
14         "query": {
15             "bool": {
16                 "should": should_clauses,
17                 "minimum_should_match": 1
18             }
19         },
20         "from": (page - 1) * page_size,
21         "size": page_size,
22         "track_total_hits": True
23     }
24     return es.search(index=INDEX_NAME, body=body)

```

示例演示

我使用温*和温?进行测试,保持账号一致,不受到个性化相关功能影响排序,可以发现温*比温?的搜索结果更多,符合我们的判断,因为*可以匹配更多字符:

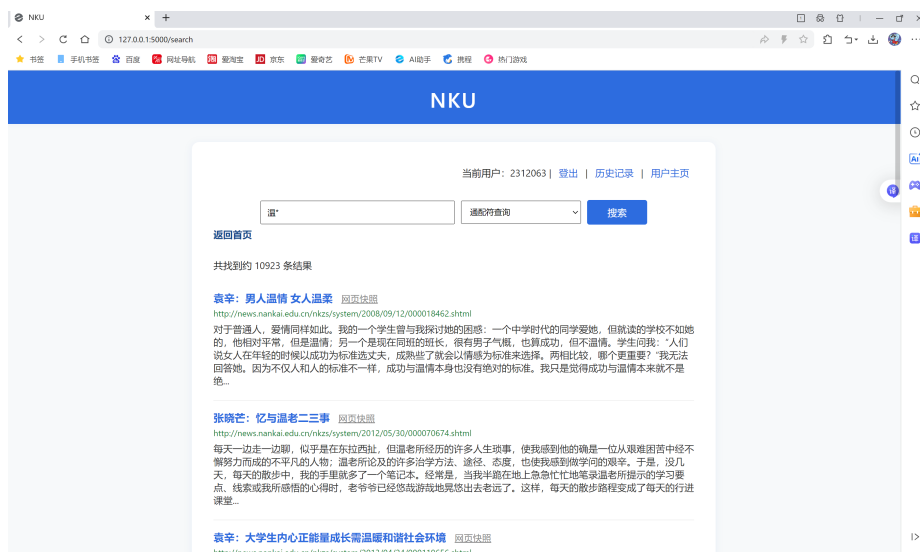


图 3.7: 通配查询

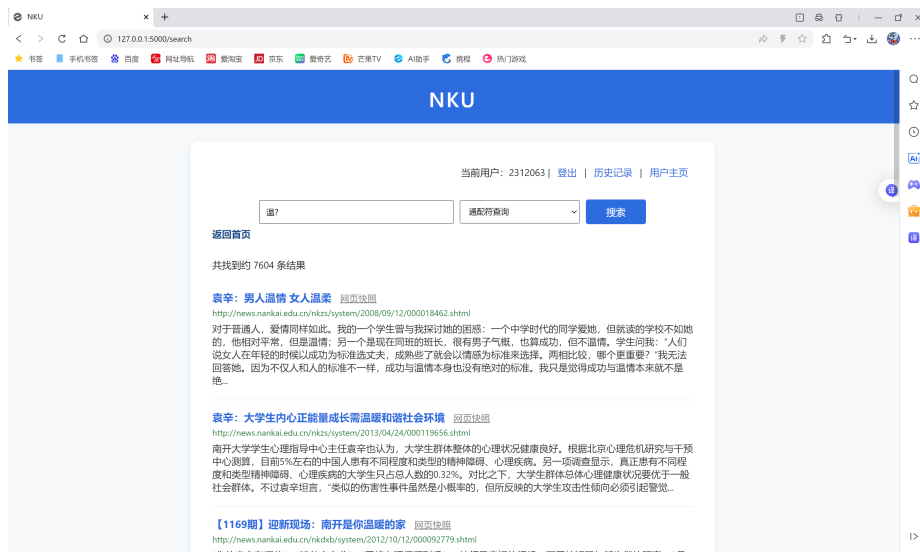


图 3.8: 通配查询

3.3.5 查询日志

查询日志功能记录用户的搜索历史,以便进行个性化推荐或分析,通过 `save_search_history` 函数进行实现:读取现有的查询日志,添加新的搜索记录,在必要时截断日志。我将用户的相关查询信息存储在 json 文件中,具体内容示例如下:

```
{ } query_log.json > { } 2
1  [
2    {
3      "query": "doc",
4      "type": "document",
5      "timestamp": "2025-06-10T22:42:18.982366"
6    },
7    {
8      "query": "\u5357*",
9      "type": "wildcard",
10     "timestamp": "2025-06-11T09:25:20.160556"
11   },
12   [
13     "query": "\u65b0\u95fb",
14     "type": "standard",
15     "timestamp": "2025-06-11T10:30:28.586903"
16   ],
17   {
18     "query": "\u5357\u5f00\u5927\u5b66\u533b\u9662",
19     "type": "standard",
20     "timestamp": "2025-06-11T10:31:18.289727"
21   },
22   {
23     "query": "\u8ba1\u7b97\u673a",
24     "type": "standard",
25     "timestamp": "2025-06-11T10:31:50.301930"
26   },
27 ]
```

代码实现

索引结构

```
1 def save_search_history(entry):
2     log_file_path = 'query_log.json'
3     logs = read_logs(log_file_path)
4     entry['timestamp'] = datetime.datetime.now().isoformat()
5     if 'user_id' in session:
6         entry['user_id'] = session['user_id']
7     logs.append(entry)
8     if len(logs) > 100:
9         logs = logs[-100:]
10    write_logs(logs, log_file_path)
```

示例演示

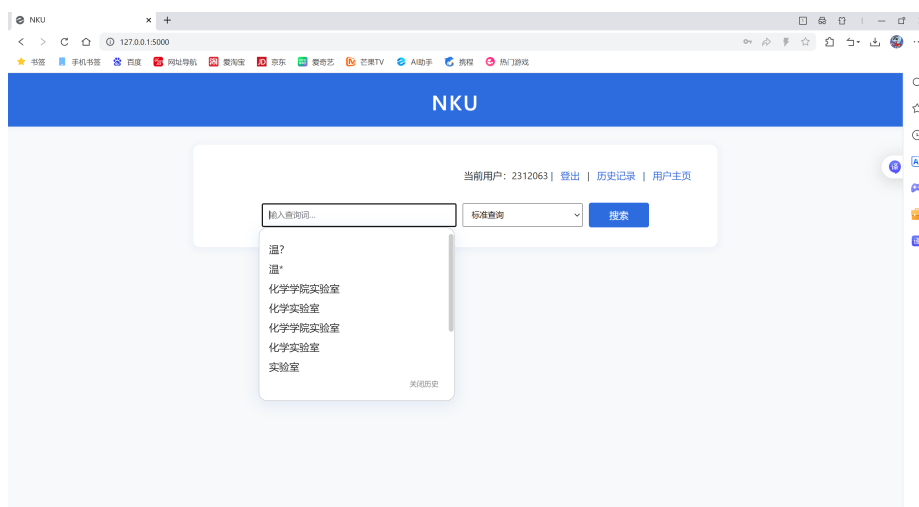


图 3.9: 主页面

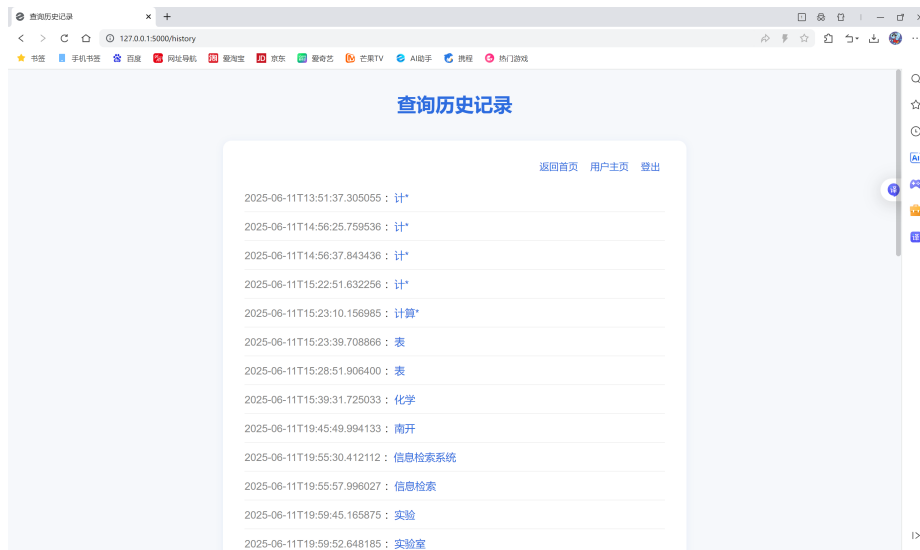


图 3.10: 历史记录

3.3.6 网页快照

网页快照功能允许用户查看搜索结果的原始 HTML 内容，即使原始网页已经发生变化或无法访问，通过 snapshot 函数进行实现：根据快照哈希值查找对应的 HTML 文件，并将其内容返回给用户

代码实现

索引结构

```

1 @app.route("/snapshot")
2 def snapshot():
3     snapshot_hash = request.args.get('snapshot_hash')
4     if not snapshot_hash:
5         return "Snapshot hash parameter is missing", 404
6     snapshot_dir = "D:/hw4/code/snapshots_1"
7     snapshot_file_path = os.path.join(snapshot_dir, f"{snapshot_hash}.html")
8     if not os.path.exists(snapshot_file_path):
9         return "No snapshot available for this hash", 404
10    snapshot_time =
11        datetime.datetime.fromtimestamp(os.path.getmtime(snapshot_file_path))
12    title = ""
13    try:
14        resp = es.search(index=INDEX_NAME, body={
15            "query": {"term": {"snapshot_hash": snapshot_hash}},
16            "_source": ["title"]
17        })
18        hits = resp.get("hits", {}).get("hits", [])
19        if hits:
20            title = hits[0]["_source"].get("title", "")
21    except Exception:
22        title = ""
23    with open(snapshot_file_path, 'r', encoding='utf-8') as file:
24        raw_html = file.read()

```

```

24     return render_template("snapshot.html",
25                             raw_html=raw_html,
26                             title=title,
27                             snapshot_time=snapshot_time.strftime("%Y-%m-%d %H:%M:%S"))

```

示例演示

我还在网页快照界面添加了一个模块展示对应的标题和获取快照的时间：



图 3.11: 网页快照

3.4 个性化查询

个性化查询的核心目标是根据用户的角色维度，如身份信息、所属学院，和搜索关键词进行综合分析，对搜索结果进行重新排序，使得更符合用户兴趣和需求的内容优先展示，实现流程包括：

- 用户注册 + 登录系统：实现基础的用户注册、登录系统，并利用 json 文件存储用户信息；
- 个性化分析：基于用户信息对搜索结果进行分析调整，根据调整对搜索结果进行重排。

代码实现

索引结构

```

1 def _calculate_boost(content, role, college, related_colleges):
2     boost = 1.0
3     if role == '教师':
4         if any(tag in content.lower() for tag in ['学术', '科研', '实验室',
5             '课题组']):
6             boost *= 1.5
7     elif role in ['本科生']:
8         if any(tag in content.lower() for tag in ['学生', '奖学金', '实习', '竞赛',
9             '社团']):
10             boost *= 1.5
11     elif role in ['研究生', '博士生']:
12         if any(tag in content.lower() for tag in ['就业', '实习', '实验', '研究',
13             '论文']):

```

```
11         boost *= 1.5
12     if college and college != '未设置':
13         if college.lower() in content.lower():
14             boost *= 1.5
15     for related_college in related_colleges:
16         if related_college.lower() in content.lower():
17             boost *= 1.2
18     return boost
```

示例演示

第一个用户为计算机学院的学生，第二个用户为非计算机学院的学生，可以发现两者的查询结果不一样。

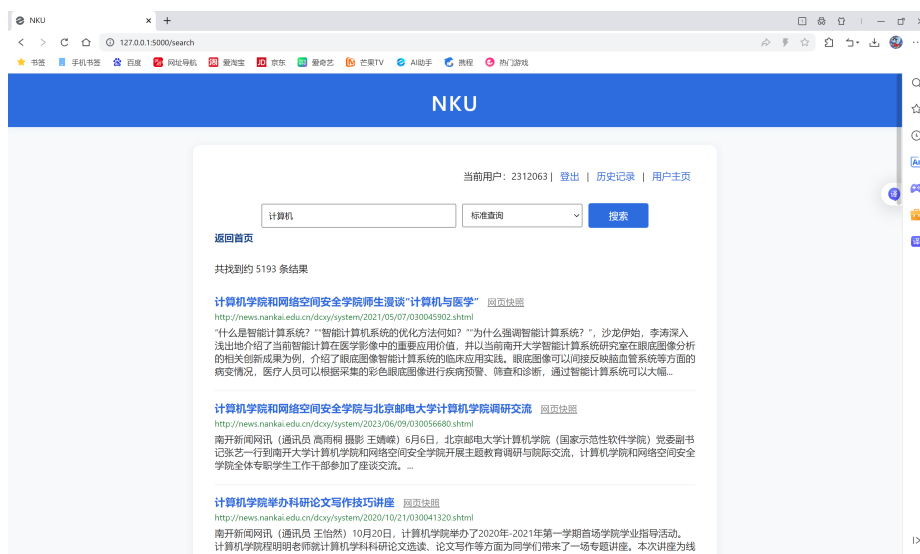


图 3.12: 个性化查询

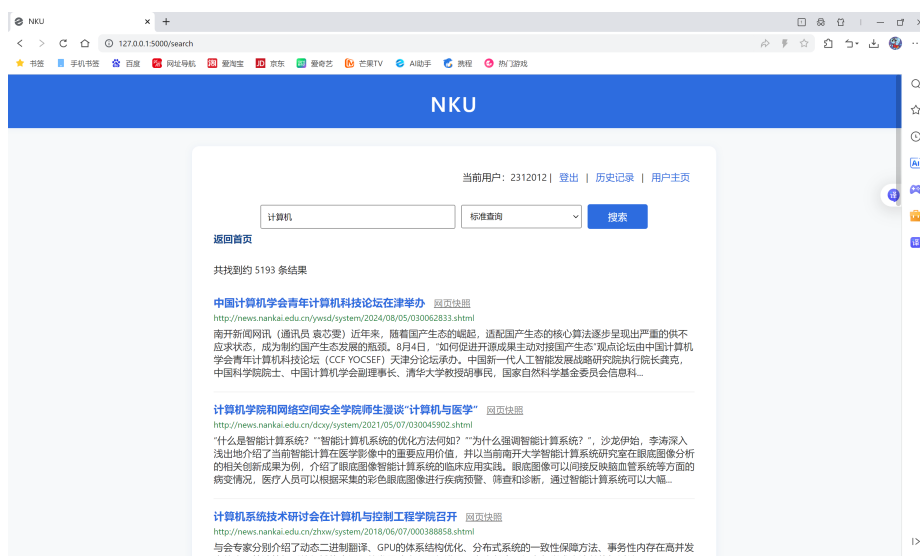


图 3.13: 个性化查询

3.5 个性化推荐

代码实现

索引结构

```
1 我实现的是搜索上的联想关联，
2 @app.route("/user_home")
3 def user_home():
4     if 'user_id' not in session:
5         return redirect(url_for('login'))
6     user_id = session['user_id']
7     log_file_path = 'query_log.json'
8     logs = read_logs(log_file_path)
9     user_logs = [log for log in logs if log.get('user_id') == user_id]
10    user_logs.sort(key=lambda x: x.get('timestamp', ''), reverse=True)
11    history_recommendations = []
12    if user_logs:
13        last_query = user_logs[0]["query"]
14        resp = site_search(last_query, page=1, page_size=10)
15        hits = resp.get("hits", {}).get("hits", [])
16        user_role = session.get('role', '')
17        user_college = session.get('college', '')
18        related_colleges = []
19        scored_recs = []
20        for hit in hits:
21            content = hit["_source"].get("content", "")
22            title = hit["_source"].get("title", "")
23            url = hit["_source"].get("url", "#")
24            boost = _calculate_boost(content, user_role, user_college,
25                                    related_colleges)
26            scored_recs.append({
27                "title": title,
28                "url": url,
29                "boost": boost
30            })
31        scored_recs.sort(key=lambda x: x["boost"], reverse=True)
32        history_recommendations = scored_recs[:5]
33    return render_template("user_home.html",
34                           user_id=user_id,
35                           history_recommendations=history_recommendations)
```

示例演示

当我输入化学的过程，搜索框会出现相关的联想关联内容，如下所示：

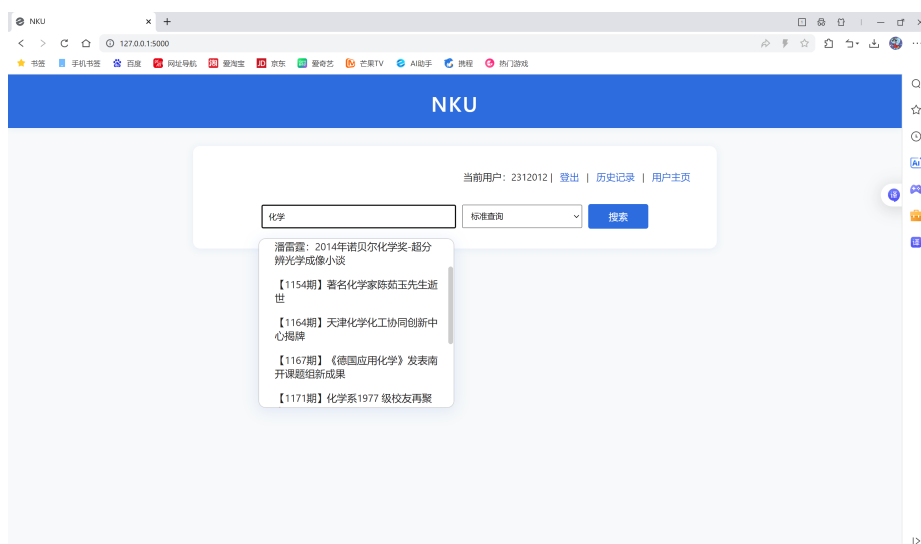


图 3.14: Caption

3.6 web 界面

3.6.1 整体架构

后端采用 Flask, 功能通过不同的路由函数分离, 如/search、/register、/login、/history、/user_home 等), 每个功能点都对应独立的前端模板 search.html、history.html、user_home.html 等;

用户信息、日志、搜索历史等数据通过 txt 和 JSON 文件存储和处理, 并通过函数如 load_users、save_users、read_logs 等进行操作。

3.6.2 核心功能模块

多样化搜索选项: /search 路由支持多种搜索类型 (普通、文档、短语、通配符), 通过 query_type 参数区分, 后端分别调用不同的搜索函数。

分页机制: 通过 page 参数和 PAGE_SIZE 常量实现分页, 便于大规模结果展示;

注册/登录/登出: /register、/login、/logout 路由实现用户注册、登录、登出, 用户信息存储于 users.json, 登录后通过 session 保存 user_id、college、role 等信息, 前端进行显示用户、下拉菜单等。

搜索历史: /history 路由展示个人搜索历史, save_search_history 函数负责日志记录

个性化推荐; /user_home 路由根据用户最近搜索历史和身份信息, 结合内容加权推荐相关结果。